

Private Service Connect - 使用 PSC 後端存取區域 Google API

來源：<https://codelabs.developers.google.com/cloudnet-psc-backend-r-gAPI?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將瞭解如何使用 Private Service Connect 存取區域性 Google API。本程式碼研究室將逐步說明如何設定 PSC 網路端點群組，並將其做為內部應用程式負載平衡器的後端。

目錄

1. 簡介
2. 程式碼研究室拓撲
3. 設定和需求
4. 事前準備
5. 建立虛擬私有雲網路、子網路和防火牆規則
6. 建立 Cloud NAT
7. 建立 Key Management 金鑰環和金鑰
8. 建立用戶端 VM 並連線至 KMS API
9. 建立 PSC NEG 和負載平衡器
10. DNS 設定
11. 透過 PSC 連線至區域性 KMS API
12. 清除步驟
13. 恭喜！

步驟 1 · 約 2 分鐘

Private Service Connect - 使用 PSC 後端存取區域性 Google API

程式碼研究室簡介

*subject*上次更新時間：3月 16, 2026

*account_circle*作者：Veda Shridharan

1. 簡介

Private Service Connect 是 Google Cloud 網路功能，可讓消費者存取供應商服務。包括透過使用者(通常是消費者) 虛擬私有雲中代管的私人端點連線至 Google API。

此外，PSC 後端可與 Google Cloud 代理負載平衡器搭配使用，指向特定區域的 Google API。使用 PSC 後端可提供更精細的消費者端控制選項，例如：

- 選擇可透過網址對應設定使用的 Google API 服務
- 更深入的觀測能力
- 整合 Cloud Armor
- 自訂網址
- 進階流量管理

若要查看可用的服務及其區域 API 清單，請按[這裡](#)。

在本程式碼研究室中，您將建立指向區域性 Cloud KMS API 的 PSC 後端，並測試與該 API 的連線。

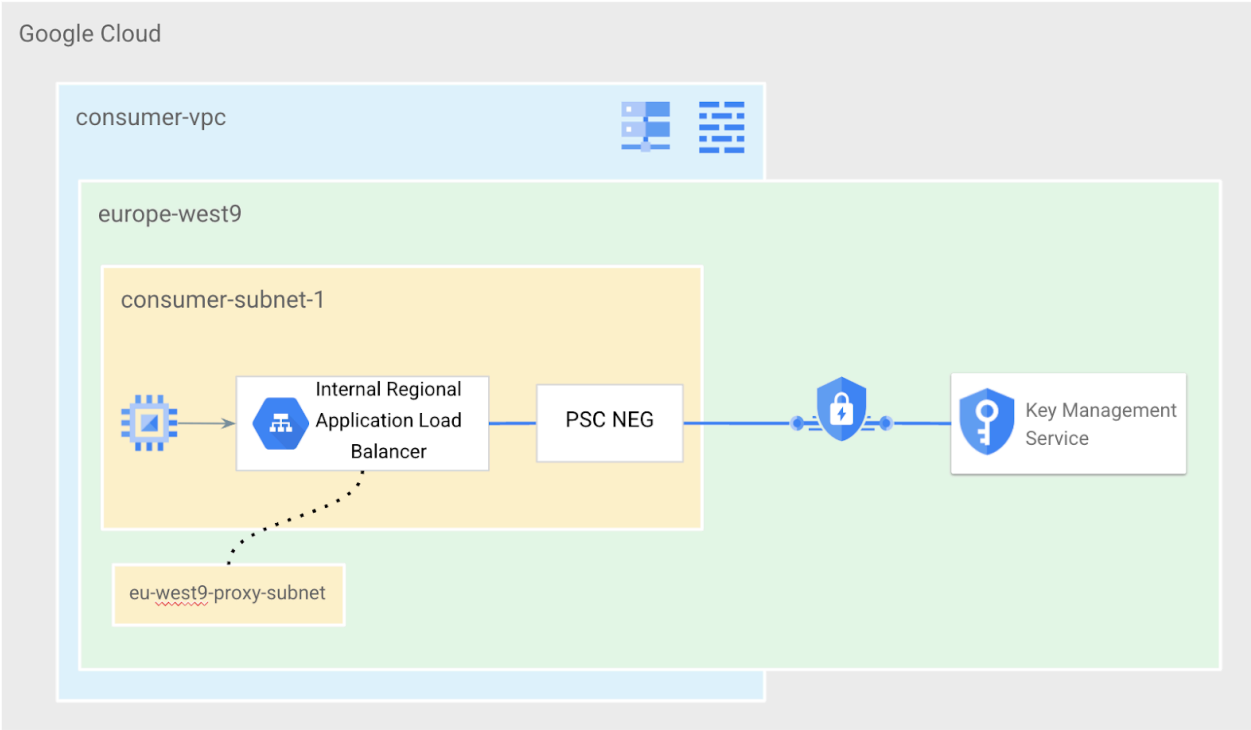
課程內容

- 建立 Cloud Key Management Service (KMS) 金鑰環和金鑰。
- 建立內部應用程式負載平衡器，並使用指向 Cloud KMS 區域性 API 的 PSC 後端
- 建立 Cloud DNS 代管私人區域和 A 記錄。
- 存取區域 Cloud KMS

軟硬體需求

- Google Cloud 專案，最好具有「擁有者」或完整的「編輯者」權限。
-

2. 程式碼研究室拓撲



系統會在 europe-west9 區域中建立一個子網路，用於代管 VM、內部區域應用程式負載平衡器轉送規則和 PSC 後端，並建立一個僅限 Proxy 的子網路，供負載平衡器使用。我們將在歐洲西部地區的 Cloud Key Management System (KMS) 中建立金鑰環和金鑰。接著，我們會建立負載平衡器和 PSC 後端，以解析 europe-west9 中的區域 KMS API。

3. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. Below the Project ID, it says 'It cannot be changed later. EDIT'. The 'Location' field is empty, and there is a 'BROWSE' button next to it. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

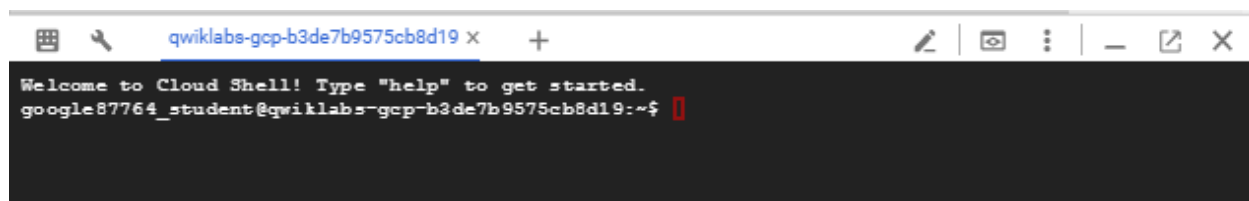
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell terminal. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19 x'. The terminal window has a black background with white text. The text reads: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a new line with the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The browser window includes standard navigation and window control buttons.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 4 · 約 2 分鐘

4. 事前準備

啟用 API

在 Cloud Shell 中，確認專案 ID 已設定完畢

```
gcloud config list project
gcloud config set project <project-id>
export PROJECT_ID=$(gcloud config get-value project)
export REGION=europe-west9
export ZONE=europe-west9-a
echo $PROJECT_ID
echo $REGION
echo $ZONE
```

啟用所有必要服務

```
gcloud services enable compute.googleapis.com
gcloud services enable servicedirectory.googleapis.com
gcloud services enable dns.googleapis.com
gcloud services enable cloudkms.googleapis.com
```

步驟 5 · 約 3 分鐘

5. 建立虛擬私有雲網路、子網路和防火牆規則

建立虛擬私有雲網路

透過 Cloud Shell

```
# Set environment variables

export VPC_NAME="consumer-vpc"
export SUBNET_NAME="consumer-subnet-1"

# Create VPC network

gcloud compute networks create ${VPC_NAME} \
  --subnet-mode=custom \
  --bgp-routing-mode=regional
```

建立子網路

透過 Cloud Shell

```
gcloud compute networks subnets create ${SUBNET_NAME} \
  --network=${VPC_NAME} \
  --region=${REGION} \
  --range=10.0.0.0/24 \
  --enable-private-ip-google-access
```

建議您在所有子網路中啟用私人 Google 存取權。

在本實驗室中，您將建立內部 L7 區域負載平衡器，指向區域 API 後端。這個負載平衡器是 Proxy 負載平衡器，因此您需要建立負載平衡器專用的「Proxy 子網路」，才能執行 Proxy。如要進一步瞭解僅限 Proxy 的子網路，請參閱[這篇文章](#)。

透過 Cloud Shell

```
gcloud compute networks subnets create eu-west9-proxy-subnet \
  --network=${VPC_NAME} \
  --region=${REGION} \
  --range=10.100.100.0/24 \
  --purpose=REGIONAL_MANAGED_PROXY \
  --role=ACTIVE
```

建立防火牆規則

在本實驗室中，您將使用 IAP 連線至所建立的執行個體。如果您不想使用 IAP，可以跳過這個步驟，改為在執行個體上新增公開 IP 位址，並建立防火牆規則，允許從 0.0.0.0/0 輸入 TCP 通訊埠 22。

如要允許 IAP 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

透過 Cloud Shell

```
gcloud compute firewall-rules create allow-ssh-iap \  
  --network=${VPC_NAME} \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

步驟 6 · 約 2 分鐘

6. 建立 Cloud NAT

您必須建立 Cloud NAT，才能下載 Linux 套件發行版本。

建立 Cloud Router

透過 Cloud Shell

```
gcloud compute routers create crnat \  
  --network=${VPC_NAME} \  
  --region=${REGION}
```

建立 Cloud NAT

透過 Cloud Shell

```
gcloud compute routers nats create europe-nat \  
  --router=crnat \  
  --auto-allocate-nat-external-ips \  
  --nat-all-subnet-ip-ranges \  
  --enable-logging \  
  --region=${REGION}
```

步驟 7 · 約 5 分鐘

7. 建立 Key Management 金鑰環和金鑰

透過 Cloud Shell

```
gcloud kms keyrings create europe-kr \  
  --location ${REGION}
```

透過 Cloud Shell

```
gcloud kms keys create europe-key \  
  --location ${REGION} \  
  --keyring europe-kr \  
  --purpose encryption
```

在 Cloud Shell 中，確認金鑰環和金鑰已在歐洲西部地區成功建立。

```
gcloud kms keys list \  
  --location ${REGION} \  
  --keyring europe-kr
```

預期結果

```
NAME: projects/<project-id>/locations/europe-west9/keyRings/europe-  
kr/cryptoKeys/europe-key  
PURPOSE: ENCRYPT_DECRYPT  
ALGORITHM: GOOGLE_SYMMETRIC_ENCRYPTION  
PROTECTION_LEVEL: SOFTWARE  
LABELS:  
PRIMARY_ID: 1  
PRIMARY_STATE: ENABLED
```

請記下金鑰的**完整路徑名稱**，因為您稍後會使用這個名稱連線。

步驟 8 · 約 10 分鐘

8. 建立用戶端 VM 並連線至 KMS API

接著，您將建立用戶端 VM、透過 SSH 連線至 VM，並測試全域 KMS API 的解析度。這項測試代表解析全球 Google API 的預設選項。

透過 Cloud Shell

```
#Create the startup script

touch startup.sh

#Open the startup.sh file using a text editor of your choice (e.g., nano, vim, gedit,
etc.)

nano startup.sh

#Paste the following script content into the startup.sh file

#!/bin/bash
sudo apt-get update
sudo apt-get install dnsutils -y
sudo apt-get install tcpdump -y

#Save the changes you made to the startup.sh file
#Use the chmod command to make the script executable

chmod +x startup.sh

#Create the VM instance

gcloud compute instances create client-vm \
  --network="${VPC_NAME}" \
  --subnet="${SUBNET_NAME}" \
  --zone="europe-west9-a" \
  --machine-type="e2-medium" \
  --no-address \
  --scopes="https://www.googleapis.com/auth/cloud-platform" \
  --image-family="debian-12" \
  --image-project="debian-cloud" \
  --metadata-from-file="startup-script=./startup.sh"
```

接下來，您需要更新預設的 Compute 服務帳戶，才能存取您建立的 KMS 金鑰。預設運算服務帳戶的格式為 `<Project_Number> -compute@developer.gserviceaccount.com`。如要取得專案編號，請從 Cloud Shell 執行下列指令，然後複製傳回結果最後一行中的編號。

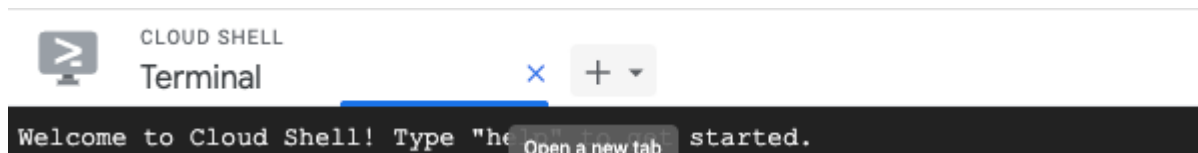
```
gcloud projects describe $PROJECT_ID
```

更新預設的 Compute 服務帳戶，取得您建立的 KMS 金鑰存取權。

透過 Cloud Shell

```
gcloud kms keys add-iam-policy-binding europe-key \
  --location $REGION \
  --keyring europe-kr \
  --member serviceAccount:<project_number>-compute@developer.gserviceaccount.com \
  --role roles/cloudkms.admin
```

點選「+」(如下方螢幕截圖所示)，建立額外的 Cloud Shell 終端機



在分頁 2 中，透過 IAP 建立通道，以 SSH 連線至 client-vm。請注意，環境變數不會傳送，您必須在下列指令中新增專案 ID。

透過 Cloud Shell

```
# Set the environment variable

export PROJECT_ID=$(gcloud config get-value project)

# ssh into the client-vm

gcloud beta compute ssh --zone europe-west9-a "client-vm" \
  --tunnel-through-iap \
  --project $PROJECT_ID
```

使用先前記下的 KMS 金鑰名稱，連線至全域 KMS API。

從分頁 2，client-vm

```
# Store the access token in a variable

TOKEN=$(gcloud auth print-access-token)

# Run the full command with maximum verbosity
curl -v \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  'https://cloudkms.googleapis.com/v1/projects/<project-id>/locations/europe-west9/keyRings/europe-kr/cryptoKeys/europe-key'
```

預期結果

```
* Trying 216.58.214.74:443...
* Connected to cloudkms.googleapis.com (216.58.214.74) port 443 (#0)
* ALPN: offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/certs/ca-certificates.crt
* CApath: /etc/ssl/certs
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384
* ALPN: server accepted h2
* Server certificate:
* subject: CN=upload.video.google.com
* start date: Aug 26 07:12:45 2024 GMT
* expire date: Nov 18 07:12:44 2024 GMT
* subjectAltName: host "cloudkms.googleapis.com" matched cert's "*.googleapis.com"
* issuer: C=US; O=Google Trust Services; CN=WR2
* SSL certificate verify ok.
* using HTTP/2
* h2h3 [:method: GET]
* h2h3 [:path: /v1/projects/<project-id>/locations/europe-west9/keyRings/europe-kr/cryptoKeys/europe-key]
* h2h3 [:scheme: https]
* h2h3 [:authority: cloudkms.googleapis.com]
* h2h3 [user-agent: curl/7.88.1]
* h2h3 [accept: /*/]
* h2h3 [authorization: Bearer dsnkjfdnvjfd]
* h2h3 [content-type: application/json]
* Using Stream ID: 1 (easy handle 0x55ed8bb7ece0)
> GET /v1/projects/<project-id>/locations/europe-west9/keyRings/europe-kr/cryptoKeys/europe-key HTTP/2
> Host: cloudkms.googleapis.com
> user-agent: curl/7.88.1
> accept: /*/
> authorization: Bearer dsnkjfdnvjfd
> content-type: application/json
>
< HTTP/2 200
< content-type: application/json; charset=UTF-8
< vary: X-Origin
< vary: Referer
< vary: Origin,Accept-Encoding
< date: Tue, 24 Sep 2024 18:25:42 GMT
< server: ESF
```

```
< cache-control: private
< x-xss-protection: 0
< x-frame-options: SAMEORIGIN
< x-content-type-options: nosniff
< accept-ranges: none
<
{
  "name": "projects/<project-id>/locations/europe-west9/keyRings/europe-
kr/cryptoKeys/europe-key",
  "primary": {
    "name": "projects/<project-id>/locations/europe-west9/keyRings/europe-
kr/cryptoKeys/europe-key/cryptoKeyVersions/1",
    "state": "ENABLED",
    "createTime": "2024-09-24T17:56:01.905156045Z",
    "protectionLevel": "SOFTWARE",
    "algorithm": "GOOGLE_SYMMETRIC_ENCRYPTION",
    "generateTime": "2024-09-24T17:56:01.905156045Z"
  },
  "purpose": "ENCRYPT_DECRYPT",
  "createTime": "2024-09-24T17:56:01.905156045Z",
  "versionTemplate": {
    "protectionLevel": "SOFTWARE",
    "algorithm": "GOOGLE_SYMMETRIC_ENCRYPTION"
  },
  "destroyScheduledDuration": "2592000s"
}
* Connection #0 to host cloudkms.googleapis.com left intact
```

檢查 DNS 將 KMS 端點解析到何處。

從 tab2，client-vm

```
dig cloudkms.googleapis.com
```

預期結果

```

<<>> DiG 9.18.28-1-deb12u2-Debian <<>> cloudkms.googleapis.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62617
;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;cloudkms.googleapis.com.      IN      A

;; ANSWER SECTION:
cloudkms.googleapis.com. 300      IN      A      142.250.74.234
cloudkms.googleapis.com. 300      IN      A      142.250.75.234
cloudkms.googleapis.com. 300      IN      A      216.58.214.170
cloudkms.googleapis.com. 300      IN      A      172.217.20.170
cloudkms.googleapis.com. 300      IN      A      172.217.20.202
cloudkms.googleapis.com. 300      IN      A      216.58.215.42
cloudkms.googleapis.com. 300      IN      A      216.58.213.74
cloudkms.googleapis.com. 300      IN      A      142.250.179.74
cloudkms.googleapis.com. 300      IN      A      142.250.179.106
cloudkms.googleapis.com. 300      IN      A      142.250.178.138
cloudkms.googleapis.com. 300      IN      A      142.250.201.170
cloudkms.googleapis.com. 300      IN      A      172.217.18.202
cloudkms.googleapis.com. 300      IN      A      216.58.214.74

;; Query time: 4 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Wed Oct 23 19:58:58 UTC 2024
;; MSG SIZE rcvd: 260

```

從 tab2 , client-vm

```
dig cloudkms.europe-west9.rep.googleapis.com
```

預期結果

```
<<>> DiG 9.18.28-1~deb12u2-Debian <<>> cloudkms.europe-west9.rep.googleapis.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2736
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;cloudkms.europe-west9.rep.googleapis.com. IN A

;; ANSWER SECTION:
cloudkms.europe-west9.rep.googleapis.com. 3043 IN A 34.1.65.232

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Wed Oct 23 20:00:04 UTC 2024
;; MSG SIZE rcvd: 85
```

Google API 的預設行為是使用 Global API 服務端點。這個端點會解析為公開 IP 清單。cloudkms.googleapis.com 的 dig 顯示了這一點。(注意：您看到的 IP 位址可能與外部 IP 位址不同。這是 Google API 的正常行為。)

透過 PSC 使用區域性 Google API 端點，您就能符合 API 流量的區域性規定，並將解析從公開變更為私人。對 cloudkms.europe-west9.rep.googleapis.com 執行的 dig 顯示，此時區域 KSM API 端點的解析結果仍為公開。

在接下來的章節中，我們會從使用 Global KMS API 端點遷移至區域端點，並使用 PSC 後端將解析度變更為私有。

步驟 9 · 約 5 分鐘

9. 建立 PSC NEG 和負載平衡器

接下來請切換回 Cloud Shell 的第一個分頁。

您將建立內部 HTTP(S) 負載平衡器，並使用指向歐洲區域 KMS 端點的網路端點群組(NEG) 做為後端服務。負載平衡器的轉送規則會做為 Private Service Connect(PSC) 端點。

建立類型為 Private Service Connect 的網路端點群組 (NEG)，並以 europe-west9-cloudkms.example.com 為目標服務

透過 Cloud Shell

```
#Set the necessary variables

NEG_NAME="l7psc-kms-neg"
PSC_TARGET="cloudkms.europe-west9.rep.googleapis.com"

#Create the Private Service Connect NEG

gcloud compute network-endpoint-groups create ${NEG_NAME} \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --network-endpoint-type=PRIVATE_SERVICE_CONNECT \
  --psc-target-service=${PSC_TARGET}

# Verify the NEG creation

gcloud compute network-endpoint-groups describe ${NEG_NAME} \
  --project=${PROJECT_ID} \
  --region=${REGION}
```

建立負載平衡器的後端服務。

透過 Cloud Shell

```
# 1. Set the necessary variables

BACKEND_SERVICE_NAME="l7-psc-kms"

# 2. Create the backend service

gcloud compute backend-services create $BACKEND_SERVICE_NAME \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --protocol=HTTPS \
  --region=$REGION \
```

將 NEG 新增至後端服務。

透過 Cloud Shell

```
gcloud compute backend-services add-backend $BACKEND_SERVICE_NAME \
  --network-endpoint-group=${NEG_NAME} \
  --region=$REGION
```

為負載平衡器建立網址對應。

透過 Cloud Shell

```
gcloud compute url-maps create l7-psc-url-map \
  --default-service=l7-psc-kms \
  --region=$REGION
```

為端點使用的自訂網址建立路徑比對器。

透過 Cloud Shell

```
gcloud compute url-maps add-path-matcher l7-psc-url-map \
  --path-matcher-name=example \
  --default-service=l7-psc-kms \
  --region=$REGION
```

為自訂網址 europe-west9-cloudkms.example.com 建立主機規則。

透過 Cloud Shell

```
gcloud compute url-maps add-host-rule l7-psc-url-map \
  --hosts=europe-west9-cloudkms.example.com \
  --path-matcher-name=example \
  --region=$REGION
```

建立負載平衡器的目標 HTTPS Proxy。這需要建立區域性 [SSL 憑證資源](#)。如需建立自行簽署憑證的步驟，請參閱[這篇文章](#)。我們將使用 openssl 建立自行簽署的憑證，並用來在 GCP 上建立區域憑證資源。目標 HTTPS Proxy 會使用這個憑證。

透過 Cloud Shell

```
# Set environment variables

export CERT_NAME="my-ssl-cert"

# Generate a private key

openssl genrsa -out private.key 2048

# Create a configuration file for the CSR

cat > csr_config.cnf << EOF
[req]
default_bits = 2048
prompt = no
default_md = sha256
req_extensions = req_ext
distinguished_name = dn

[dn]
CN = example.com
O = My Organization
C = US

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = example.com
DNS.2 = *.example.com
EOF

# Create a CSR using the configuration

openssl req -new -key private.key -out server.csr -config csr_config.cnf

# Create a self-signed certificate using the CSR

openssl x509 -req -days 365 -in server.csr -signkey private.key -out server.crt \
    -extfile csr_config.cnf -extensions req_ext

# Verify the certificate

openssl x509 -in server.crt -text -noout

#Create a regional SSL certificate resource

gcloud compute ssl-certificates create ${CERT_NAME} \
    --region=${REGION} \
```

```
--certificate=server.crt \  
--private-key=private.key
```

#Create the target HTTPS proxy for the load balancer

```
gcloud compute target-https-proxies create psc-http-proxy \  
  --region=${REGION} \  
  --url-map=l7-psc-url-map \  
  --ssl-certificates=${CERT_NAME}
```

為負載平衡器建立轉送規則，做為 Private Service Connect 端點。轉送規則的 IP 位址必須屬於 VPC 中的區域子網路，且該子網路與您存取的 API 端點位於相同區域。

透過 Cloud Shell

```
# Set environment variables
```

```
export PROXY_NAME="psc-http-proxy"
```

```
# Create the forwarding rule
```

```
gcloud compute forwarding-rules create kms-lb-rule \  
  --project=${PROJECT_ID} \  
  --region=${REGION} \  
  --load-balancing-scheme=INTERNAL_MANAGED \  
  --network=${VPC_NAME} \  
  --subnet=${SUBNET_NAME} \  
  --target-https-proxy=${PROXY_NAME} \  
  --target-https-proxy-region=${REGION} \  
  --address=10.0.0.100 \  
  --ports=443
```

10. DNS 設定

在本節中，您將為 example.com 建立私人 DNS 區域，以及指向上一個步驟中建立轉送規則的 A 記錄。

建立代管 DNS 私人區域。

透過 Cloud Shell

```
# Set environment variables

export LB_RULE_NAME="kms-lb-rule"
export DNS_ZONE_NAME="example-com-private-zone"

# Create the private DNS zone

gcloud dns managed-zones create ${DNS_ZONE_NAME} \
  --dns-name="example.com." \
  --description="Private DNS zone for example.com" \
  --visibility=private \
  --networks=${VPC_NAME}
```

為 europe-west9-cloudkms.example.com 建立 A 記錄。

透過 Cloud Shell

```
gcloud dns record-sets transaction start \
  --zone=${DNS_ZONE_NAME}

gcloud dns record-sets transaction add 10.0.0.100 \
  --name=europe-west9-cloudkms.example.com \
  --ttl=300 \
  --type=A \
  --zone=${DNS_ZONE_NAME}

gcloud dns record-sets transaction execute \
  --zone=${DNS_ZONE_NAME}
```

11. 透過 PSC 連線至區域性 KMS API

切換回分頁 2 中的 **client-vm**，執行 tcpdump 查看所有連線詳細資料，並透過 PSC 後端測試與區域 KMS 端點的連線。

```
sudo tcpdump -i any net 10.0.0.100 or port 53 -n
```

在 **Cloud Shell** 中開啟第 3 個分頁，然後透過 **SSH** 連線至 **client-vm**。

```
# Set environment variables

export PROJECT_ID=$(gcloud config get-value project)
export KMS_HOSTNAME="europe-west9-cloudkms.example.com"
export KEY_RING="europe-kr"
export KEY_NAME="europe-key"
export REGION="europe-west9"

# Command to access the specific key

curl -k
"https://${KMS_HOSTNAME}/v1/projects/${PROJECT_ID}/locations/${REGION}/keyRings/${KEY_RING}/cryptoKeys/${KEY_NAME}" \
-H "Authorization: Bearer $(gcloud auth print-access-token)"
```

curl 指令 + TCPDUMP 的預期結果

```
{
  "name": "projects/<project-id>/locations/europe-west9/keyRings/europe-kr/cryptoKeys/europe-key",
  "primary": {
    "name": "projects/<project-id>/locations/europe-west9/keyRings/europe-kr/cryptoKeys/europe-key/cryptoKeyVersions/1",
    "state": "ENABLED",
    "createTime": "2024-10-10T18:50:24.357027036Z",
    "protectionLevel": "SOFTWARE",
    "algorithm": "GOOGLE_SYMMETRIC_ENCRYPTION",
    "generateTime": "2024-10-10T18:50:24.357027036Z"
  },
  "purpose": "ENCRYPT_DECRYPT",
  "createTime": "2024-10-10T18:50:24.357027036Z",
  "versionTemplate": {
    "protectionLevel": "SOFTWARE",
    "algorithm": "GOOGLE_SYMMETRIC_ENCRYPTION"
  },
  "destroyScheduledDuration": "2592000s"
}
```

Tcpdump output

```
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
18:13:51.220209 lo      In   IP 127.0.0.1.48873 > 127.0.0.53.53: 2086+ [1au] A? europe-west9-cloudkms.example.com. (62)
18:13:51.220230 lo      In   IP 127.0.0.1.48873 > 127.0.0.53.53: 24619+ [1au] AAAA? europe-west9-cloudkms.example.com. (62)
18:13:51.220669 ens4    Out  IP 10.0.0.2.52121 > 169.254.169.254.53: 8885+ [1au] A? europe-west9-cloudkms.example.com. (62)
18:13:51.220784 ens4    Out  IP 10.0.0.2.53041 > 169.254.169.254.53: 57748+ [1au] AAAA? europe-west9-cloudkms.example.com. (62)
18:13:51.229638 ens4    In   IP 169.254.169.254.53 > 10.0.0.2.52121: 8885 1/0/1 A 10.0.0.100 (78)
18:13:51.229945 lo      In   IP 127.0.0.53.53 > 127.0.0.1.48873: 2086 1/0/1 A 10.0.0.100 (78)
18:13:51.230068 ens4    In   IP 169.254.169.254.53 > 10.0.0.2.53041: 57748 0/1/1 (155)
18:13:51.230203 lo      In   IP 127.0.0.53.53 > 127.0.0.1.48873: 24619 0/1/1 (155)
18:13:51.230390 ens4    Out  IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [S], seq 1606150798, win 65320, options [mss 1420,sackOK,TS val 4135800856 ecr 0,nop,wscale 7], length 0
18:13:51.232565 ens4    In   IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [S.], seq 1041507402, ack 1606150799, win 65535, options [mss 1420,sackOK,TS val 2276748382 ecr 4135800856,nop,wscale 8], length 0
18:13:51.232583 ens4    Out  IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [.], ack 1, win 511, options [nop,nop,TS val 4135800859 ecr 2276748382], length 0
```

```
18:13:51.235494 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [P.], seq 1:518,
ack 1, win 511, options [nop,nop,TS val 4135800862 ecr 2276748382], length 517
18:13:51.236571 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [.], ack 518, win
267, options [nop,nop,TS val 2276748387 ecr 4135800862], length 0
18:13:51.239119 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [P.], seq 1:1356,
ack 518, win 267, options [nop,nop,TS val 2276748389 ecr 4135800862], length 1355
18:13:51.239140 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [.], ack 1356,
win 501, options [nop,nop,TS val 4135800865 ecr 2276748389], length 0
18:13:51.240978 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [P.], seq
518:598, ack 1356, win 501, options [nop,nop,TS val 4135800867 ecr 2276748389],
length 80
18:13:51.241266 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [P.], seq
598:684, ack 1356, win 501, options [nop,nop,TS val 4135800867 ecr 2276748389],
length 86
18:13:51.241366 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [P.], seq
684:1646, ack 1356, win 501, options [nop,nop,TS val 4135800867 ecr 2276748389],
length 962
18:13:51.242370 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [.], ack 684, win
267, options [nop,nop,TS val 2276748392 ecr 4135800867], length 0
18:13:51.242619 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [P.], seq
1356:1433, ack 1646, win 278, options [nop,nop,TS val 2276748393 ecr 4135800867],
length 77
18:13:51.242730 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [P.], seq
1646:1677, ack 1433, win 501, options [nop,nop,TS val 4135800869 ecr 2276748393],
length 31
18:13:51.248724 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [.], ack 1677,
win 278, options [nop,nop,TS val 2276748399 ecr 4135800869], length 0
18:13:51.296676 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [P.], seq
1433:2357, ack 1677, win 278, options [nop,nop,TS val 2276748447 ecr 4135800869],
length 924
18:13:51.297223 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [F.], seq 1677,
ack 2357, win 501, options [nop,nop,TS val 4135800923 ecr 2276748447], length 0
18:13:51.298304 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [P.], seq
2357:2381, ack 1678, win 278, options [nop,nop,TS val 2276748448 ecr 4135800923],
length 24
18:13:51.298305 ens4 In IP 10.0.0.100.443 > 10.0.0.2.59474: Flags [F.], seq 2381,
ack 1678, win 278, options [nop,nop,TS val 2276748448 ecr 4135800923], length 0
18:13:51.298336 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [R], seq
1606152476, win 0, length 0
18:13:51.298343 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [R], seq
1606152476, win 0, length 0
```

返回分頁 2 視窗，檢查 tcpdump 資訊。您會發現，透過建立的 PSC 端點，您能夠存取 Cloud KMS 區域端點，而且 europe-west9 區域端點會以私密方式解析為您建立的 Managed Cloud DNS 區域。tcpdump 輸出中的相關行已在上方醒目顯示，並在下方列出：

```
18:13:51.229638 ens4 In IP 169.254.169.254.53 > 10.0.0.2.52121: 8885 1/0/1 A 10.0.0.100 (78)
```

(GCP 中繼資料伺服器會傳回 A 記錄：10.0.0.100 - 負載平衡器 IP。DNS 解析作業正常運作。europa-west9-cloudkms.example.com 會解析為 10.0.0.100，也就是負載平衡器 IP)

```
18:13:51.230390 ens4 Out IP 10.0.0.2.59474 > 10.0.0.100.443: Flags [S], seq 1606150798, win 65320, options [mss 1420,sackOK,TS val 4135800856 ecr 0,nop,wscale 7], length 0 (這會顯示與負載平衡器 IP 建立 HTTPS 連線的 TCP 交握)
```

在第 3 個分頁中，用戶端 VM

```
dig europe-west9-cloudkms.example.com
```

預期結果

```
; <<>> DiG 9.18.28-1~deb12u2-Debian <<>> europe-west9-cloudkms.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7008
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;europe-west9-cloudkms.example.com. IN A

;; ANSWER SECTION:
europe-west9-cloudkms.example.com. 300 IN A      10.0.0.100

;; Query time: 12 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Fri Oct 11 20:03:00 UTC 2024
;; MSG SIZE rcvd: 78
```

dig 指令的輸出內容顯示，我們為 europe-west9-cloudkms.example.com 建立的自訂網址已正確解析為 10.0.0.100，也就是內部負載平衡器的 IP。對 europe-west9-cloudkms.example.com 的要求會導向內部負載平衡器，然後透過 Private Service Connect 轉送至 KMS 區域端點。

現在可以關閉 client-vm 的兩個 SSH 分頁。

步驟 12 · 約 5 分鐘

12. 清除步驟

從單一 Cloud Shell 終端機刪除實驗室元件

```
# Set environment variables

export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --
format="value(projectNumber)")
export REGION=europe-west9
export ZONE=europe-west9-a
export VPC_NAME="consumer-vpc"
export SUBNET_NAME="consumer-subnet-1"
export NEG_NAME="l7psc-kms-neg"
export BACKEND_SERVICE_NAME="l7-psc-kms"
export CERT_NAME="my-ssl-cert"
export PROXY_NAME="psc-http-proxy"
export LB_RULE_NAME="kms-lb-rule"
export DNS_ZONE_NAME="example-com-private-zone"

# Delete DNS records and zone

gcloud dns record-sets delete europe-west9-cloudkms.example.com. \
  --zone=${DNS_ZONE_NAME} --type=A --quiet
gcloud dns managed-zones delete ${DNS_ZONE_NAME} --quiet

# Delete Load Balancer components

gcloud compute forwarding-rules delete ${LB_RULE_NAME} --region=${REGION} --quiet
gcloud compute target-https-proxies delete ${PROXY_NAME} --region=${REGION} --quiet
gcloud compute url-maps delete l7-psc-url-map --region=${REGION} --quiet
gcloud compute backend-services delete ${BACKEND_SERVICE_NAME} --region=${REGION} --
quiet
gcloud compute network-endpoint-groups delete ${NEG_NAME} --region=${REGION} --quiet

# Delete SSL certificate

gcloud compute ssl-certificates delete ${CERT_NAME} --region=${REGION} --quiet

# Delete VM instance

gcloud compute instances delete client-vm --zone=${ZONE} --quiet

# Delete firewall rules

gcloud compute firewall-rules delete allow-ssh-iap --quiet

# Delete Cloud NAT and router

gcloud compute routers nats delete europe-nat --router=crnat --region=${REGION} --
quiet
gcloud compute routers delete crnat --region=${REGION} --quiet
```

```
# Delete subnets and VPC

gcloud compute networks subnets delete ${SUBNET_NAME} --region=${REGION} --quiet
gcloud compute networks subnets delete eu-west9-proxy-subnet --region=${REGION} --quiet
gcloud compute networks delete ${VPC_NAME} --quiet

# Schedule KMS key for deletion and provide instructions for keyring deletion

gcloud kms keys remove-iam-policy-binding europe-key \
  --location ${REGION} \
  --keyring europe-kr \
  --member serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com \
  --role roles/cloudkms.admin

gcloud kms keys versions destroy 1 --location=${REGION} --keyring=europe-kr --key=europe-key

# Disable services (optional, only if you want to completely disable these APIs)

gcloud services disable compute.googleapis.com --force
gcloud services disable servicedirectory.googleapis.com --force
gcloud services disable dns.googleapis.com --force
gcloud services disable cloudkms.googleapis.com --force

# Clean up local files

rm -f private.key server.csr server.crt csr_config.cnf startup.sh
```

步驟 13 · 約 0 分鐘

13. 恭喜！

恭喜您完成本程式碼研究室。
